TECHNICKÁ UNIVERZITA
Neurónové siete 2025/2026

Detekcia sieťových útokov pomocou neurónových sietí

Projektová správa

2025/2026

Obsah

1 Formulácia problému **3**

2 Teoretický background **4**
2.1 Multilayer Perceptron (MLP) . . . 4
2.2 Long Short-Term Memory (LSTM) . . . 4
2.3 Regularizácia a stabilizácia tréningu . . . 5

3 Popis datasetu a predspracovanie dát **5**
3.1 CIC-IDS2017 . . . 5
3.1.1 Predspracovanie dát . . . 5
3.1.2 Rozdelenie datasetu . . . 6
3.2 NSL-KDD . . . 6
3.2.1 Predspracovanie dát . . . 6

4 Metodológia **7**
4.1 MLP Architektúry . . . 7
4.1.1 Pre dataset CIC-IDS2017 . . . 7
4.1.2 Pre dataset NSL-KDD . . . 8
4.2 LSTM Architektúry . . . 9
4.2.1 Pre dataset CIC-IDS2017 . . . 9
4.2.2 Pre dataset NSL-KDD . . . 10
4.3 Infraštruktúra — HPC Perun . . . 10
4.4 Riešenie triednej nerovnováhy . . . 11
4.5 Evaluačné metriky a ich zdôvodnenie . . . 12

5 Návrh experimentov **12**
5.1 Zdôvodnenie rôznych architektúr medzi fázami . . . 13
5.2 Ablačná štúdia — zdôvodnenie . . . 15

6 Výsledky **15**
6.1 Výsledky na CIC-IDS2017 . . . 15
6.2 Výsledky na NSL-KDD . . . 16
6.3 Súhrnné porovnanie všetkých experimentov . . . 17
6.4 Detailná analýza výsledkov . . . 17
6.5 Interpretácia výsledkov . . . 17

7 Kritická analýza a diskusia **18**
7.1 Porovnávanie architektúr 18
7.2 Analýza výsledkov ablačnej štúdie 20
7.3 Analýza chýb 21
7.4 Technické poznatky z HPC nasadenia 22

8 Obmedzenia a možnosti ďalšej práce **23**
8.1 Obmedzenia 23
8.2 Možnosti ďalšej práce 23

9 Príspevok jednotlivých členov tímu **24**

10 Zoznam použitej literatúry **24**

1 Formulácia problému

Detekcia sieťových prienikov (Network Intrusion Detection System – NIDS) je jednou z najdôležitejších úloh v oblasti kybernetickej bezpečnosti. Cieľom NIDS je v reálnom čase identifikovať škodlivú aktivitu v počítačovej sieti – či už ide o pokus o neoprávnený prístup, distribuované odmietnutie služby (DDoS), skenovanie portov, šírenie malvéru alebo iný typ útoku.

Tradičné priemyselné riešenia sú založené na dvoch hlavných princípoch:

1. **Detekcia založená na signatúrach (signature-based):** Porovnáva sieťový prenos s databázou známych vzorov (signatúr) útokov. Tento prístup je presný na známe útoky, ale úplne zlyháva pri nových, doteraz neznámych typoch útokov (zero-day exploits).

2. **Detekcia založená na anomáliách (anomaly-based):** Vytvára model normálneho správania siete a akákoľvek odchýlka je považovaná za potenciálny útok. Hoci dokáže odhaliť nové útoky, trpí vysokým počtom falošných poplachov (false positives), čo ju robí ťažko nasaditeľnou v praxi.

Moderné prístupy sa preto čoraz viac prikláňajú k metódam strojového a hlbokého učenia, ktoré dokážu automaticky extrahovať diskriminačné vzory z dát bez potreby ručne definovaných pravidiel. Táto práca skúma, či neurónové siete – konkrétne viacvrstvový perceptrón (MLP) a dlhá krátkodobá pamäť (LSTM) – dokážu spoľahlivo rozlíšiť útočný a legitímny sieťový prenos na základe štatistických príznakov sieťových tokov.

Problém je formulovaný ako **binárna klasifikácia**:

- Trieda 0 (Benign): legitímna sieťová komunikácia (napr. bežné webové prehliadanie, e-mail, FTP).

- Trieda 1 (Attack): akýkoľvek typ sieťového útoku (DoS, DDoS, Bruteforce, Infiltration, Botnet, Port Scan, Web Attack atď.).

Hlavné výzvy, ktoré musí model riešiť, sú:

1. **Výrazná triedna nerovnováha:** V reálnych sieťach aj v použitých datasetoch (CIC-IDS2017, NSL-KDD) tvorí legitímny prenos približne 80–90% všetkých vzoriek. Model trénovaný na nevyvážených dátach má tendenciu klasifikovať takmer všetko ako "Benign", čo vedie k vysokej miere zmeškaných útokov (false negatives).

2. **Vysoká dimenzionalita:** Každý sieťový tok je opísaný 78 (CIC-IDS2017) až 41 (NSL-KDD) príznakmi – zahŕňajú základné charakteristiky (trvanie, počet paketov, počet bajtov), štatistiky dĺžok paketov, flagy TCP, počty chybových paketov a pod.

3. **Cenová asymetria chýb:** V bezpečnostných aplikáciách nie sú všetky chyby rovnako závažné. Zmeškaný útok (False Negative) je oveľa nebezpečnejší ako falošný poplach (False Positive).

4. **Časový kontext (pre LSTM):** Niektoré typy útokov (napr. pomalé skenovanie, botnetová komunikácia) nie sú identifikovateľné z jedného izolovaného sieťového toku, ale až z postupnosti viacerých tokov.

5. **Rozdielne charakteristiky datasetov:** Zatiaľ čo CIC-IDS2017 je moderný, rozsiahly dataset (2,8 milióna vzoriek), NSL-KDD je starší benchmarkový dataset (125 tisíc tréningových vzoriek) s inou distribúciou príznakov a typov útokov.

Cieľom tejto práce je navrhnúť, implementovať a experimentálne zhodnotiť rôzne architektúry neurónových sietí (MLP, LSTM) na dvoch odlišných datasetoch, pričom sa zameriavame nielen na celkovú presnosť, ale najmä na minimalizáciu zmeškaných útokov a schopnosť generalizácie na neznáme typy útokov. Súčasťou je aj ablačná štúdia, ktorá skúma vplyv kapacity modelu a dĺžky časového okna na výsledný výkon.

2 Teoretický background

2.1 Multilayer Perceptron (MLP)

MLP je základná architektúra doprednej neurónovej siete skladajúca sa z plne prepojených vrstiev. Pre každý neurón v skrytej vrstve platí: $y = f(Wx + b)$, kde W sú váhy, b je bias a f je aktivačná funkcia. V tejto práci používame ReLU (Rectified Linear Unit) pre skryté vrstvy a Sigmoid pre výstupnú vrstvu binárnej klasifikácie.

MLP slúži ako baseline model — jeho jednoduché usporiadanie umožňuje rýchly tréning a interpretáciu, no nezachytáva časové vzťahy medzi paketmi. Pri spracovaní sekvencií (napr. sieťovej komunikácie) by MLP musel dostať na vstup celé časové okno naraz, čo neumožňuje dynamické prispôsobovanie sa postupne prichádzajúcim dátam a ignoruje závislosti medzi paketmi vzdialenými v čase.

2.2 Long Short-Term Memory (LSTM)

Motivácia: Problém zanikajúceho gradientu

Rekurentné neurónové siete (RNN) majú teoreticky schopnosť uchovávať informácie o minulých udalostiach v podobe aktivácií (tzv. krátkodobá pamäť). V praxi však trénovanie RNN metódami spätného šírenia chyby v čase (Backpropagation Through Time, BPTT) naráža na zásadný problém: pri spätnom šírení chybových signálov do minulosti dochádza k ich exponenciálnemu zosilneniu alebo, častejšie, k exponenciálnemu zániku. Hochreiter (1991) vo svojej diplomovej práci analyticky ukázal, že veľkosť gradientu pri spätnom prechode cez q časových krokov je ohraničená výrazom $n(f'_{\max}|W|_A)^q$, kde $f'_{\max}$ je maximálna derivácia aktivačnej funkcie (napr. 0.25 pre sigmoid) a $|W|_A$ je norma matice váh. Pre bežné hodnoty váh (absolútna hodnota < 4) tak gradient s rastúcim q rýchlo klesá k nule. To znemožňuje sieti naučiť sa závislosti, ktoré vyžadujú uchovanie informácie cez viac než 10 časových krokov.

Architektúra LSTM a konštantný tok chyby

Hochreiter a Schmidhuber (1997) navrhli architektúru nazvanú Long Short-Term Memory (LSTM), ktorá tento problém rieši zavedením špecializovaných pamäťových buniek (memory cells). Každá bunka obsahuje *constant error carousel* (CEC) – lineárnu jednotku so samospätnou väzbou s váhou 1.0. Vďaka tomu môže chybový signál prúdiť cez vnútorný stav bunky $s_{c_j}(t)$ s konštantnou veľkosťou, bez zániku či explózie.

Aby sa však zabránilo rušeniu uloženej informácie nežiaducimi vstupmi a výstupmi, LSTM zavádza dve multiplikatívne brány (gate units):

- **Vstupná brána** in_j – riadi, kedy sa nová informácia zapíše do pamäťovej bunky. Tým sa rieši *input weight conflict* – rovnaké vstupné váhy by inak museli súčasne slúžiť na zapamätanie aj ignorovanie rôznych vstupov.
- **Výstupná brána** out_j – riadi, kedy sa obsah bunky sprístupní zvyšku siete. Výstupná brána rieši *output weight conflict* – chráni ostatné jednotky pred rušivými vplyvmi pamäťovej bunky v čase, keď jej obsah nie je potrebný.

Formálne, pre vnútorný stav bunky c_j^v platí:

$$s_{c_j^v}(t) = s_{c_j^v}(t-1) + y^{in_j}(t) \cdot g(net_{c_j^v}(t)), \qquad y^{c_j^v}(t) = y^{out_j}(t) \cdot h\big(s_{c_j^v}(t)\big).$$

Tréning LSTM využíva modifikovanú verziu BPTT s *truncated backpropagation* – chybové signály sa nešíria ďalej do minulosti, keď opustia brány alebo vstup bunky. To zaisťuje, že vnútri CEC zostáva tok chyby konštantný, a zároveň je výpočtová zložitosť na časový krok $O(W)$, kde W je počet váh – rovnako ako pri BPTT, ale na rozdiel od RTRL ($O(W^2)$).

LSTM v kontexte detekcie sieťových útokov

Sieťová komunikácia je prirodzene sekvenčná: útočník vykonáva skenovanie, odosiela škodlivé payloady, následne nadväzuje spojenie – tieto udalosti sú rozložené v čase. LSTM vďaka svojej schopnosti uchovávať informáciu po stovky časových krokov dokáže modelovať správanie útočníka aj v prípade, že medzi podozrivým paketom a jeho následkom uplynie dlhý čas. Navyše, LSTM je robustná voči šumu a distribuovaným reprezentáciám – čo je kľúčové pre reálne dáta ako CIC-IDS2017, ktoré obsahujú veľa náhodných alebo irelevantných paketov (distractor symbols) zmiešaných s útočnými vzormi.

2.3 Regularizácia a stabilizácia tréningu

Dropout náhodne deaktivuje určitý podiel neurónov počas tréningu, čím zabraňuje overfittingu. Batch Normalization normalizuje aktivácie každej vrstvy, čo stabilizuje tréning a umožňuje vyššie learning rates. Oba mechanizmy sú obzvlášť dôležité pri práci s CIC-IDS2017, ktorý obsahuje veľké množstvo vzoriek s výraznými hodnotami príznakov.

Pri LSTM sa často používa aj *gradient clipping* (orezanie gradientov) na potlačenie prípadných explózií, a *inicializácia biasov výstupných brán na záporné hodnoty* (napr. -2, -4). Záporná počiatočná hodnota výstupnej brány potláča aktivitu pamäťových buniek na začiatku tréningu, čím sa predchádza *abuse problem* – sieť sa nespolieha na pamäťové bunky ako na obyčajné bias jednotky.

3 Popis datasetu a predspracovanie dát

3.1 CIC-IDS2017

Dataset CIC-IDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) bol vytvorený Univerzitou v New Brunswicku. Obsahuje zachytené sieťové toky z reálnych scenárov útokov realizovaných v kontrolovanom prostredí počas 5 pracovných dní.

Vlastnosť	Popis
Počet záznamov	∼2,8 milióna sieťových tokov
Po čistení (NaN/Inf)	2 827 876 vzoriek (odstránených 2 867)
Počet príznakov	78 → 69 po odstránení konštantných stĺpcov
Triedy	Benign: 2 271 320 (80.3%) | Attack: 556 556 (19.7%)
Typy útokov	DoS, DDoS, Brute Force, XSS, SQL Injection, Infiltration, Botnet, Port Scan
Formát	8 CSV súborov (Monday–Friday, ∼170k–693k riadkov/súbor)

Tabuľka 1: Charakteristika datasetu CIC-IDS2017

3.1.1 Predspracovanie dát

Pipeline predspracovania pozostáva z nasledujúcich krokov:

- Načítanie a zlúčenie: všetky CSV súbory z priečinka MachineLearningCVE sú načítané a konkatenované do jedného DataFrame.
- Čistenie názvov stĺpcov: odstránenie nadbytočných medzier zo záhlaví (typický problém CIC-IDS2017).
- Náhrada nekonečných hodnôt: hodnoty +inf a -inf sú nahradené NaN a následne riadky s NaN sú odstránené. Príznaky ako `Flow Bytes/s` môžu obsahovať inf pri nulových časoch.

- Odstránenie duplikátnych stĺpcov: stĺpec `Fwd Header Length.1` je duplikátom a je vymazaný.
- Binárizácia náveští: BENIGN → 0, akýkoľvek útok → 1.
- Odstránenie konštantných príznakov: príznaky s nulovou variabilitou nenesú informáciu (odstránených 8 stĺpcov).
- Normalizácia MinMaxScaler: škálovanie všetkých príznakov do rozsahu [0, 1].

3.1.2 Rozdelenie datasetu

Množina	Podiel	Veľkosť (MLP)	Účel
Tréningová	64%	1 809 840	Učenie váh modelu
Validačná	16%	452 460	Ladenie hyperparametrov
Testovacia	20%	565 576	Finálna nezaujatá evaluácia

Tabuľka 2: Rozdelenie datasetu CIC-IDS2017

Pre MLP modely bolo použité stratifikované náhodné rozdelenie (`shuffle=True`). Pre LSTM modely bol implementovaný *per-day stratifikovaný split* — z každého dňa (CSV súboru) sa nezávisle vyčlenilo 64 %/16 %/20 % riadkov v chronologickom poradí. Tým každá množina obsahuje záznamy zo všetkých dní a teda všetkých typov útokov, pričom temporálny poriadok vnútri každého dňa je zachovaný.

3.2 NSL-KDD

Dataset NSL-KDD je štandardným benchmarkom v oblasti detekcie sieťových prienikov. Ide o vylepšenú verziu pôvodného datasetu KDD'99, z ktorého boli odstránené redundantné záznamy, čím sa zabránilo skresleniu pri učení modelov. Tento dataset neslúži na simuláciu "Big Data" (ako CIC-IDS2017), ale primárne na overenie robustnosti a generalizačných schopností architektúr na štandardizovaných dátach.

Vlastnosť	Popis
Počet tréningových záznamov	125 973 (súbor `KDDTrain+.csv`)
Počet testovacích záznamov	22 544 (súbor `KDDTest+.csv`)
Počet príznakov	41 (38 numerických, 3 kategorické)
Typy útokov	DoS, Probe, R2L (Root to Local), U2R (User to Root)
Triedna nerovnováha (Test)	∼43% Benign (9 710), ∼57% Attack (12 833)
Formát	Predpripravené oddelené CSV súbory

Tabuľka 3: Charakteristika datasetu NSL-KDD

3.2.1 Predspracovanie dát

Pipeline predspracovania pre NSL-KDD sa mierne líši z dôvodu prítomnosti kategorických premenných:

- Načítanie stĺpcov: Názvy stĺpcov boli extrahované zo súboru `Field Names.csv` a priradené k dátam, keďže pôvodné CSV súbory neobsahujú hlavičku.
- Odstránenie nepotrebných atribútov: Stĺpec `difficulty` bol vymazaný, keďže neslúži ako predikčný príznak.

- Binárizácia náveští: Záznamy s náveštím `normal` → 0, akýkoľvek iný typ útoku → 1.
- Kódovanie kategorických príznakov: Textové atribúty (`protocol_type`, `service`, `flag`) boli transformované na číselné hodnoty pomocou `LabelEncoder`. Kódovanie prebehlo na spojených dátach (train + test), aby sa predišlo chybám z neznámych kategórií v testovacej množine.
- Normalizácia: Použitie `MinMaxScaler` na preškálovanie všetkých numerických hodnôt do intervalu [0, 1].
- Generovanie časových okien (pre LSTM): dáta transformované do 3D tenzorov s veľkosťou okna (*window size*) = 5 záznamov (`create_windows` funkcia).

Špecifikom testovacej množiny NSL-KDD je, že obsahuje špecifické typy útokov, ktoré sa v tréningovej množine nenachádzajú. Tento fakt robí evaluáciu na tomto datasete obzvlášť prísnou pre testovanie generalizačných schopností neurónových sietí.

4 Metodológia

4.1 MLP Architektúry

4.1.1 Pre dataset CIC-IDS2017

Architektúra MLP Baseline je navrhnutá ako kompaktná, ale dostatočne expresívna sieť pre vysokodimenzionálne tabuľkové dáta. Skladá sa z nasledujúcich vrstiev:

$$\text{Input}(69) \to \text{Dense}(64, \text{ReLU}) \to \text{BatchNorm} \to \text{Dropout}(0.2) \to \text{Dense}(32, \text{ReLU}) \to \text{Dense}(1, \text{Sigmoid}) \tag{1}$$

Vstupná vrstva. Po predspracovaní má každý sieťový tok dimenziu $d = 69$ príznakov. Vstupná vrstva teda prijíma vektor $\mathbf{x} \in [0, 1]^{69}$ pre každú vzorku nezávisle — MLP neuvažuje žiadny sekvenčný kontext medzi po sebe idúcimi tokmi.

Prvá skrytá vrstva — Dense(64, ReLU). Plne prepojená vrstva s 64 neurónmi a aktivačnou funkciou ReLU:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{64\times 69} \tag{2}$$

ReLU ($f(z) = \max(0, z)$) bola zvolená pre svoju výpočtovú jednoduchosť a efektívne potlačenie problému zanikajúceho gradientu v porovnaní so sigmoid/tanh aktiváciami v skrytých vrstvách.

Batch Normalization. Po prvej skrytej vrstve je zaradená vrstva Batch Normalization, ktorá normalizuje aktivácie po každom mini-batchi:

$$\hat{h}_{1,i} = \frac{h_{1,i} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \cdot \gamma + \beta \tag{3}$$

kde $\mu_{\mathcal{B}}$ a $\sigma_{\mathcal{B}}^2$ sú priemer a rozptyl mini-batchu, γ a β sú trénovateľné parametre škálovania a posunutia, $\varepsilon = 10^{-3}$ zabraňuje deleniu nulou. Tento mechanizmus je obzvlášť dôležitý pre CIC-IDS2017, kde niektoré príznaky (napr. `Flow Bytes/s`, `Packet Length Std`) nadobúdajú extrémne hodnoty aj po normalizácii, čo by bez BatchNorm destabilizovalo tréning.

Dropout(0.2). Regularizačná vrstva s pravdepodobnosťou deaktivácie $p = 0.2$ náhodne vypína 20 % neurónov počas každého tréningového kroku. Tým sa predchádza ko-adaptácii neurónov a overfittingu na tréningovú množinu. Počas inferencie sú všetky neuróny aktívne a ich výstupy sú škálované faktorom $(1 - p)$.

Druhá skrytá vrstva — Dense(32, ReLU). Druhá vrstva redukuje reprezentáciu na 32 neurónov:

$$\mathbf{h}_2 = \mathrm{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{32\times 64} \tag{4}$$

Postupné zužovanie dimenzie ($69 \to 64 \to 32$) núti sieť extrahovať čoraz abstraktnejšie reprezentácie príznakov sieťového toku.

Výstupná vrstva — Dense(1, Sigmoid).

$$\hat{y} = \sigma(\mathbf{w}_{\text{out}}^\top \mathbf{h}_2 + b_{\text{out}}) = \frac{1}{1 + e^{-(\mathbf{w}_{\text{out}}^\top \mathbf{h}_2 + b_{\text{out}})}} \in (0, 1) \tag{5}$$

Binárna predikcia sa určí prahovaním: $\hat{c} = \mathbf{1}[\hat{y} > 0.5]$.

Optimalizátor a stratová funkcia. Model je trénovaný optimalizátorom Adam s predvolenou rýchlosťou učenia $\eta = 10^{-3}$ a binárnou krížovou entropiou ako stratovou funkciou:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_{c_i}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{6}$$

kde w_{c_i} je váha triedy vzorky i (Benign: $w_0 = 0.623$, Attack: $w_1 = 2.541$).

Celkový počet parametrov.

$$(69{\cdot}64{+}64){+}(64{\cdot}4{\cdot}2){+}(64{\cdot}32{+}32){+}(32{+}1) = 4{,}545{+}512{+}2{,}080{+}33 = 7{,}170 \text{ parametrov} \tag{7}$$

4.1.2 Pre dataset NSL-KDD

Pre menší benchmarkový dataset NSL-KDD používame zjednodušenú verziu MLP bez Batch Normalization. Táto voľba zámerne testuje, či sieť dokáže generalizovať bez silnej regularizácie aktivácií, a zároveň reflektuje nižšiu komplexitu a menší rozsah datasetu (125 973 tréningových vzoriek oproti 1 809 840 pri CIC-IDS2017). Architektúra má tvar:

$$\mathrm{Input}(41) \to \mathrm{Dense}(64, \mathrm{ReLU}) \to \mathrm{Dense}(32, \mathrm{ReLU}) \to \mathrm{Dense}(1, \mathrm{Sigmoid}) \tag{8}$$

Vstupná vrstva. NSL-KDD obsahuje 41 príznakov: 38 numerických a 3 kategorické (`protocol_type`, `service`, `flag`). Kategorické príznaky boli pred vstupom do siete zakódované pomocou `LabelEncoder` — kódovanie prebehlo na spojených dátach (train + test), aby sa predišlo chybám pri výskyte neznámych kategórií v testovacej množine. Vstupný vektor je $\mathbf{x} \in [0, 1]^{41}$.

Prečo bez Batch Normalization. Pri CIC-IDS2017 bola BatchNorm nevyhnutná z dôvodu extrémnych hodnôt príznakov (napr. `Flow Bytes/s` s hodnotami v rádoch 10^6) a veľkého počtu vzoriek per mini-batch, kde štatistiky batchu sú stabilné. NSL-KDD má výrazne menší dataset a menej extrémne distribúcie príznakov po normalizácii, preto BatchNorm nepriniesla merateľné zlepšenie v predbežných testoch a bola vynechaná v záujme jednoduchosti modelu. Dropout nebol použitý ani z rovnakého dôvodu — menší dataset s nižšou dimenziou príznakov nevykazoval výrazné príznaky overfittingu počas tréningu.

Prvá skrytá vrstva — Dense(64, ReLU).

$$\mathbf{h}_1 = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{64\times 41} \tag{9}$$

Napriek menšiemu vstupu ($d = 41$) zachovávame rovnaký počet neurónov (64) ako pri CIC-IDS2017 architektúre. Tým umožňujeme sieti vytvoriť dostatočne bohatú latentú reprezentáciu, keďže NSL-KDD obsahuje štyri kategórie útokov (DoS, Probe, R2L, U2R) s výrazne odlišnými štatistickými profilmi.

Celkový počet parametrov.

$$(41 \cdot 64 + 64) + (64 \cdot 32 + 32) + (32 \cdot 1 + 1) = 2{,}688 + 2{,}080 + 33 = 4{,}801 \text{ parametrov} \tag{10}$$

Model je teda takmer o polovicu menší ako jeho CIC-IDS2017 ekvivalent (4 801 vs. 7 170 parametrov), čo zodpovedá nižšej vstupnej dimenzii a absencii BatchNorm parametrov.

Špecifikum testovacej množiny NSL-KDD. Kľúčovou vlastnosťou tohto benchmarku je, že testovacia množina (`KDDTest+.csv`) obsahuje typy útokov, ktoré sa v tréningovej množine vôbec nevyskytujú. Tento fakt priamo vysvetľuje nižší Recall (0.63) dosiahnutý na testovacej množine.

4.2 LSTM Architektúry

4.2.1 Pre dataset CIC-IDS2017

Na rozdiel od MLP, ktorý klasifikuje každý sieťový tok ako izolovanú vzorku, LSTM model spracúva *sekvencie* po sebe idúcich tokov a dokáže zachytiť temporálne vzory útočného správania rozložené v čase. Architektúra má tvar:

$$\begin{aligned} \text{Input}(10,\, 69) &\to \text{LSTM}(64,\, \text{return_seq=True}) \to \text{BatchNorm} \to \text{Dropout}(0.2) \\ &\to \text{LSTM}(32) \to \text{Dense}(16,\, \text{ReLU}) \to \text{Dense}(1,\, \text{Sigmoid}) \end{aligned} \tag{11}$$

Vstup — časové okná. Namiesto jednotlivých vektorov $\mathbf{x} \in \mathbb{R}^{69}$ prijíma LSTM trojrozmerný tenzor $\mathbf{X} \in \mathbb{R}^{B \times T \times F}$, kde B je veľkosť mini-batchu, $T = 10$ je dĺžka časového okna a $F = 69$ je počet príznakov. Každá vzorka v batchi teda predstavuje postupnosť 10 po sebe idúcich sieťových tokov, pričom mítka $y \in \{0, 1\}$ patrí poslednému toku v okne (index $T - 1$).

Časové okná sú generované funkciou `create_sequences`: pre každý index $i \in \{0, \ldots, N - T\}$ sa vytvorí dvojica $(\mathbf{X}_{i:i+T},\, y_{i+T-1})$.

Motivácia pre sekvenčný prístup. Niektoré typy útokov nie sú identifikovateľné z jedného izolovaného toku, ale až z ich postupnosti:

- **Port Scan:** útočník odosiela pakety na rôzne porty v krátkom časovom intervale — jednotlivý tok vyzerá legitímne, ale séria tokov odhaľuje systematické skenovanie.
- **Botnet:** pravidelná, rytmická komunikácia s C&C serverom sa prejavuje ako opakujúci sa vzor v časovej osi.
- **Infiltration:** viacfázový útok (reconnaissance → exploitation → exfiltration) zanecháva stopy rozložené cez desiatky až stovky tokov.
- **Pomalý DoS (Slow Loris):** nízkoobratový útok, ktorý nepresiahne prahy jednoduchých pravidiel, ale jeho dlhodobá prítomnosť je viditeľná v časovej sekvencii.

Prvá LSTM vrstva — LSTM(64, return_sequences=True). LSTM bunka v čase t prijíma vstup $\mathbf{x}_t \in \mathbb{R}^{69}$ a predchádzajúci skrytý stav $\mathbf{h}_{t-1} \in \mathbb{R}^{64}$, pričom aktualizuje svoj vnútorný stav podľa rovníc:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \qquad \text{(forget gate)} \tag{12}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \qquad \text{(input gate)} \tag{13}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \qquad \text{(candidate cell)} \tag{14}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \qquad \text{(cell state update)} \tag{15}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \qquad \text{(output gate)} \tag{16}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \qquad \text{(hidden state)} \tag{17}$$

kde $\odot$ označuje Hadamardov (prvkový) súčin a σ je sigmoická funkcia. Nastavenie `return_sequences=True` znamená, že prvá LSTM vrstva vracia celú sekvenciu skrytých stavov $[\mathbf{h}_1, \ldots, \mathbf{h}_{10}]$, čím umožňuje druhej LSTM vrstve spracovať celú temporálnu históriu.

Počet parametrov prvej LSTM vrstvy:

$$4 \times \big[(69 + 64) \times 64 + 64\big] = 4 \times 8\,576 = 34\,304 \tag{18}$$

Druhá LSTM vrstva — LSTM(32, return_sequences=False). Druhá vrstva s 32 bunkami vracia iba posledný skrytý stav $\mathbf{h}_{10} \in \mathbb{R}^{32}$, ktorý reprezentuje komprimovanú temporálnu sumarizáciu celej 10-krokovej sekvencie.

Počet parametrov druhej LSTM vrstvy:

$$4 \times \big[(64 + 32) \times 32 + 32\big] = 4 \times 3\,104 = 12\,416 \tag{19}$$

Celkový počet parametrov.

$$\underbrace{34\,304}_{\text{LSTM}_1} + \underbrace{512}_{\text{BatchNorm}} + \underbrace{12\,416}_{\text{LSTM}_2} + \underbrace{528}_{\text{Dense(16)}} + \underbrace{17}_{\text{Dense(1)}} = 49\,777 \text{ parametrov} \tag{20}$$

4.2.2 Pre dataset NSL-KDD

Pre benchmarkový dataset NSL-KDD bola zvolená zjednodušená LSTM architektúra s jednou rekurentnou vrstvou a kratším časovým oknom:

$$\text{Input}(5, 41) \rightarrow \text{LSTM}(64, \text{return_seq=False}) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(1, \text{Sigmoid}) \tag{21}$$

Rozdiel oproti CIC-IDS2017 variante je zámerný a vychádza z troch faktorov:

- **Veľkosť datasetu:** NSL-KDD obsahuje 125 973 tréningových vzoriek oproti 1 809 840 pri CIC-IDS2017 — teda iba ∼7 % objemu. Hlbšia architektúra by pri takomto množstve dát rýchlo overfitovala tréningovú množinu.
- **Dimenzia príznakov:** 41 príznakov (oproti 69) predstavuje jednoduchší vstupný priestor.
- **Povaha datasetu:** NSL-KDD neobsahuje explicitné časové razítka ani garantovaný chronologický poriadok záznamov — sekvencie window=5 sú skôr lokálnym kontextom než skutočnou temporálnou históriou.

Počet parametrov LSTM vrstvy: $4 \times \big[(41 + 64) \times 64 + 64\big] = 27\,136$. Celkovo: $27\,136 + 65 = 27\,201$ parametrov.

Model	Parametre	Pomer vs. MLP Baseline
MLP Baseline (CIC)	7 170	1.0×
MLP Benchmark (NSL)	4 801	0.67×
LSTM Benchmark (NSL)	27 201	3.8×
LSTM Advanced (CIC)	49 777	6.9×

Tabuľka 4: Porovnanie počtu parametrov všetkých architektúr

4.3 Infraštruktúra — HPC Perun

Experimenty boli spustené na superpočítači HPC Perun (SLURM cluster) s nasledujúcou konfiguráciou:

- GPU: NVIDIA H200 (140 831 MB VRAM, Compute Capability 9.0)

- CUDA: verzia 12.9, cuDNN: 9.2.1.0
- TensorFlow: 2.20.0, Keras: 3.10.0
- Prostredie: Python 3.9, pip lokálna inštalácia (`tensorflow[and-cuda]`)
- Tréningový čas MLP: ∼5 s/epocha | LSTM: ∼24 s/epocha
- Celkový čas behu: ∼15 minút (18:53–19:08, 1. mája 2026)

4.4 Riešenie triednej nerovnováhy

Oba datasety vykazujú výraznú nerovnováhu tried, ktorá by bez korekcie spôsobila, že model by sa naučil prediktovať majoritnú triedu (Benign/Normal) a ignorovať minoritnú (Attack).

Dataset	Benign/Normal	Attack	Pomer	Imbalance ratio
CIC-IDS2017 (train)	1 457 644	352 196	80.3 % / 19.7 %	4.1 : 1
NSL-KDD (train)	67 343	58 630	53.5 % / 46.5 %	1.1 : 1
NSL-KDD (test)	9 710	12 833	43.1 % / 56.9 %	1 : 1.3

Tabuľka 5: Distribúcia tried v použitých datasetoch

Distribúcia tried v datasetoch.

Matematická formulácia váh tried. Váhy tried sú vypočítané podľa vzorca implementovaného v `sklearn.utils.class_weight.compute_class_weight`:

$$w_c = \frac{N}{K \cdot N_c}, \quad c \in \{0, 1\} \tag{22}$$

kde N je celkový počet tréningových vzoriek, $K = 2$ je počet tried a N_c je počet vzoriek triedy c. Pre CIC-IDS2017:

$$w_0\,(\text{Benign}) = \frac{1\,809\,840}{2 \times 1\,457\,644} = 0.621 \tag{23}$$

$$w_1\,(\text{Attack}) = \frac{1\,809\,840}{2 \times 352\,196} = 2.571 \tag{24}$$

Pre NSL-KDD: $w_0 = 0.935$, $w_1 = 1.074$ — takmer symetrické okolo 1.0, čo potvrdzuje takmer vyváženú distribúciu tréningových dát tohto benchmarku.

Vplyv na stratovú funkciu.

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] \tag{25}$$

Pre CIC-IDS2017 to znamená, že každá vzorka útoku ($w_1 = 2.571$) prispieva do straty $2.571/0.621 \approx 4.1\times$ viac ako vzorka legitímnej komunikácie. Metóda váh tried bola zvolená ako optimálny kompromis — nepridáva ani neodoberá vzorky, nevyžaduje zmenu architektúry ani predspracovania a je priamo integrovaná do Keras API.

Vplyv na namerané výsledky. Bez váh tried by modely na CIC-IDS2017 pravdepodobne klasifikovali všetky vzorky ako Benign — dosiahli by Accuracy $\approx 80.3\,\%$ pri Recall(Attack) ≈ 0. Použitie váh tried zabezpečilo, že MLP Baseline dosiahol Recall(Attack) = 1.00 s FNR = 0.47 %.

4.5 Evaluačné metriky a ich zdôvodnenie

Voľba evaluačných metrík je v oblasti detekcie sieťových útokov rovnako dôležitá ako samotná architektúra modelu. Celková presnosť (Accuracy) je pri nerovnomerne rozdelených triedach zavádzajúca — model predikujúci výhradne triedu Benign by dosiahol Accuracy = 80.3 % na CIC-IDS2017 pri nulovej schopnosti detekcie útokov.

		Predikovaná trieda	
		Benign (0)	**Attack (1)**
Skutočná trieda	**Benign (0)**	TN	FP
	Attack (1)	FN	TP

Tabuľka 6: Matica zámen pre binárnu klasifikáciu útokov

Základné pojmy — matica zámen. V bezpečnostnom kontexte je FN výrazne nebezpečnejší ako FP — neodhalený útočník môže spôsobiť reálne škody, zatiaľ čo falošný poplach iba zbytočne zaťažuje administrátorov.

Prehľad metrík.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{26}$$

$$\text{F1} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}, \qquad \text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} = 1 - \text{Recall} \tag{27}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \qquad \text{AUC} = P(\hat{y}_{\text{attack}} > \hat{y}_{\text{benign}}) \tag{28}$$

FNR (False Negative Rate) je primárnou bezpečnostnou metrikou — každý prípad FN predstavuje útok, ktorý prešiel detekčným systémom bez povšimnutia. **FPR** (False Positive Rate) degraduje použiteľnosť systému v praxi — administrátori sú zahltení falošnými poplachmi a skutočné hrozby môžu byť prehliadnuté v šume (*alert fatigue*). **ROC-AUC** je nezávislá od prahu rozhodnutia a od distribúcie tried. **Average Precision (AP)** — plocha pod Precision-Recall krivkou — je citlivejšia na výkon modelu pri minoritnej triede ako ROC-AUC.

5 Návrh experimentov

Experimenty sú rozdelené do dvoch fáz podľa použitého datasetu. Fáza 1 využíva CIC-IDS2017 pre analýzu v prostredí veľkých dát na GPU infraštruktúre HPC Perun, Fáza 2 využíva NSL-KDD pre benchmarkové overenie generalizačných schopností architektúr na štandardizovanom datasete s neznámymi typmi útokov v testovacej množine.

Experiment	Model	Konfigurácia	Cieľ
Fáza 1: CIC-IDS2017 — 1 809 840 tréningových vzoriek, 69 príznakov, GPU NVIDIA H200			
E1: MLP Baseline	MLP	dropout=0.2, units=(64,32), BatchNorm, epochs=15, batch=256	Referenčná línia bez sekvenčného kontextu. Každý tok klasifikovaný nezávisle.
E2: MLP Large (ablation)	MLP	dropout=0.4, units=(128,64), BatchNorm, epochs=15, batch=256	Vplyv kapacity siete a sily regularizácie. Porovnanie s E1 izoluje efekt zdvojnásobenia neurónov a zvýšenia Dropout.
E3: LSTM Advanced	LSTM	window=10, lstm=(64,32), BatchNorm, Dropout=0.2, epochs=15, batch=256, per-day split	Využitie sekvenčného kontextu 10 po sebe idúcich tokov. Porovnanie s E1/E2 izoluje prínos temporálnej informácie.
E4: LSTM Window=5 (ablation)	LSTM	window=5, lstm=(64,32), BatchNorm, Dropout=0.2, epochs=15, batch=256, per-day split	Vplyv dĺžky kontextového okna. Skrátenie z 10 na 5 znižuje pamäťové nároky.
Fáza 2: NSL-KDD — 125 973 tréningových vzoriek, 41 príznakov, CPU			
E5: Benchmark MLP	MLP	units=(64,32), bez BatchNorm, epochs=10, batch=128	Test schopnosti modelu na štandardnom benchmarku s neznámymi útokmi v testovacej množine.
E6: Benchmark LSTM	LSTM	window=5, lstm=(64), Dropout=0.2, epochs=10, batch=128	Test sekvenčnej analýzy na i.i.d. datasete bez skutočného časového poriadku.

Tabuľka 7: Komplexný návrh experimentov

5.1 Zdôvodnenie rôznych architektúr medzi fázami

Rozdiely v architektúrach medzi Fázou 1 a Fázou 2 sú zámerné a vychádzajú z kvantitatívnych aj kvalitatívnych rozdielov medzi datasetmi.

Dataset + Model	Tréning. vzorky	Parametre	Pomer	Riziko
CIC-IDS2017 + MLP Baseline	1 809 840	7 170	252 000 : 1	Nízke
CIC-IDS2017 + LSTM Advanced	1 809 840	49 777	36 000 : 1	Nízke
NSL-KDD + MLP Benchmark	125 973	4 801	26 000 : 1	Stredné
NSL-KDD + LSTM Benchmark	125 968	27 201	4 600 : 1	Vyššie

Tabuľka 8: Pomer tréningových vzoriek k počtu parametrov modelu

Objem dát a riziko overfittingu.

BatchNormalization — kedy je nevyhnutná. BatchNormalization bola zaradená do architektúr pre CIC-IDS2017, ale vynechaná pre NSL-KDD. Dôvody sú dvojaké:

- **Distribúcia hodnôt príznakov:** CIC-IDS2017 obsahuje príznaky s extrémne zošikmenými distribúciami aj po MinMax normalizácii — napr. `Flow Bytes/s` a `Packet Length Std` majú dlhé pravostranné chvosty spôsobené DDoS útokmi. NSL-KDD má po normalizácii oveľa rovnomernejšie distribúcie.
- **Hĺbka siete:** Pri dvoch LSTM vrstvách (CIC-IDS2017) dochádza ku kaskádovým posuvom distribúcií aktivácií (*internal covariate shift*), ktoré BatchNorm koriguje. Jednovrstvový LSTM (NSL-KDD) tento problém nevykazuje v rovnakej miere.

Rozdelenie datasetu — shuffle vs. per-day stratifikovaný split. Voľba metodológie rozdelenia datasetu predstavuje jeden z najvýznamnejších technických poznatkov tejto práce — priamo ovplyvňuje výsledky LSTM modelov o desiatky percentuálnych bodov.

Pre **MLP modely** na oboch datasetoch bolo použité náhodné stratifikované premiešanie (`shuffle=True, stratify=y`), čo zabezpečuje rovnomerné zastúpenie tried v každej množine a stabilné gradientové aktualizácie.

Pre **LSTM modely** boli testované tri prístupy:

1. **Shuffle=True:** náhodné premiešanie eliminuje temporálne závislosti — okno 10 susedných vzoriek obsahuje náhodne vybrané toky bez časového kontextu. Experimentálne overené: ROC-AUC ≈ 0.5, TP $= 0$. *Nepoužiteľné pre LSTM.*
2. **Globálny chronologický split (shuffle=False):** tréningová množina obsahuje *výhradne* DoS a Brute Force útoky (pondelok–streda), testovacia množina WebAttacks, Botnet a PortScan (štvrtok–piatok). Výsledkom je domain shift: ROC-AUC $= 0.9029$, val_loss rástol z 0.093 na 1.635. *Nepoužiteľné bez ďalších úprav.*
3. **Per-day stratifikovaný split (zvolená metóda):** z každého dňa (CSV súboru) sa nezávisle vyčlení 64 %/16 %/20 % riadkov v chronologickom poradí. Každá množina tak obsahuje záznamy zo *všetkých* dní a teda všetkých typov útokov. Výsledok: ROC-AUC $= 0.9821$, stabilná konvergencia.

Metóda splitu	**ROC-AUC**	**F1**	**FNR**
Shuffle=True	≈ 0.500	≈ 0	$\approx 100\,\%$
Globálny chrono. split	0.9029	0.681	$\sim 21\,\%$
Per-day split (finálny)	**0.9821**	**0.9477**	$\sim$ **9** %
MLP Baseline (shuffle=True)	0.9985	0.9504	0.47 %

Tabuľka 9: Vplyv metodológie rozdelenia datasetu na výkon LSTM Advanced (E3)

Rozdiel medzi globálnym a per-day splitom (ΔROC-AUC $= +0.079$) je väčší ako rozdiel medzi MLP a LSTM pri rovnakom splite (ΔROC-AUC $= 0.016$). Toto potvrdzuje, že *metodológia rozdelenia dát má väčší vplyv na výsledky ako voľba architektúry* — fundamentálny poznatok pre budúce experimenty na scenárových datasetoch typu CIC-IDS2017.

Vlastnosť	CIC-IDS2017	NSL-KDD
Vstupná dimenzia	69 príznakov	41 príznakov
Hĺbka LSTM	2 vrstvy (64, 32)	1 vrstva (64)
BatchNormalization	Áno	Nie
Dropout	0.2 (LSTM, MLP base) / 0.4 (MLP Large)	0.2 (LSTM)
Časové okno	10 (base), 5 (ablat.)	5
Split pre LSTM	Per-day stratifikovaný	Náhodný
Počet epoch	15	10
Batch size	256	128
Hardvér	NVIDIA H200 (GPU)	CPU

Tabuľka 10: Porovnanie architektonických rozhodnutí pre oba datasety

Zhrnutie architektonických rozdielov.

5.2 Ablačná štúdia — zdôvodnenie

Ablačná štúdia systematicky vyhodnocuje vplyv troch kľúčových faktorov na výkon detekcie útokov pri zachovaní všetkých ostatných podmienok konštantných (dataset, optimalizátor, počet epoch, batch size).

Vplyv kapacity modelu (E1 vs. E2). Experiment E2 (MLP Large) oproti E1 (MLP Baseline) mení výhradne dve hyperparametre: počet neurónov $(64, 32) \rightarrow (128, 64)$ a mieru Dropout $0.2 \rightarrow 0.4$. Hypotézy pred experimentom:

- **H1 (underfitting):** Ak E2 výrazne prekoná E1, základná architektúra nestačí zachytiť komplexitu 69-dimenzionálneho vstupného priestoru CIC-IDS2017.
- **H2 (saturácia):** Ak sú výsledky porovnateľné alebo E2 dosiahne horšie výsledky, model E1 dosiahol saturačný bod a väčší Dropout (0.4) nadmerne obmedzuje kapacitu učenia.

Vplyv temporálneho kontextu (E1/E2 vs. E3). Porovnaním LSTM (E3) s MLP modelmi (E1, E2) izolujeme čistý prínos sekvenčných informácií pri zachovaní porovnateľnej kapacity siete (49 777 vs. 7 170 parametrov). Ak LSTM dosiahne porovnateľné výsledky ako MLP, potvrdzuje to, že temporálny kontext prináša merateľnú hodnotu v podobe dramatického zníženia FPR.

Vplyv veľkosti okna (E3 vs. E4). Skrátenie okna z 10 na 5 záznamov testuje kompromis medzi dĺžkou temporálneho kontextu, pamäťovou náročnosťou a počtom generovaných sekvencií. Kratšie okno produkuje viac tréningových sekvencií ($N-5$ vs. $N-10$), ale môže stratiť dlhodobé závislosti typické pre pomalé, viacfázové útoky.

6 Výsledky

6.1 Výsledky na CIC-IDS2017

Experimenty boli spustené na GPU NVIDIA H200 (HPC Perun, 1. mája 2026, 18:53–19:08). Testovacia množina obsahuje 565 575 vzoriek (523 421 Benign, 42 154 Attack). Nižší podiel útokov v testovacej množine (7.5 % oproti 19.7 % celkovo) je dôsledkom per-day stratifikovaného splitu — pondelkový súbor (529 918 riadkov, výhradne BENIGN) prispieva do testovacej množiny iba legitímnymi tokmi.

Model	Accuracy	F1 (Attack)	ROC-AUC	FNR	Avg. Prec.
E1: MLP Baseline	0.98	0.9504	0.9985	0.47 %	0.9941
E2: MLP Large	0.98	0.9490	0.9986	0.37 %	0.9942
E3: LSTM Win=10	0.99	0.9477	0.9821	∼9 %	0.9482
E4: LSTM Win=5	0.99	0.9455	0.9825	9.10 %	0.9450

Tabuľka 11: Výsledky na testovacej množine CIC-IDS2017 — per-day stratifikovaný split, 1. mája 2026

Model	TP	TN	FP	FN	FNR
E1: MLP Baseline	110 783	442 710	11 555	528	0.47 %
E2: MLP Large	110 903	443 439	10 826	408	0.37 %
E3: LSTM Win=10	∼38 400	∼522 900	∼500	∼3 750	∼9 %
E4: LSTM Win=5	38 316	522 840	581	3 838	9.10 %

Tabuľka 12: Matica zámen na testovacej množine CIC-IDS2017

Detailná analýza chýb — CIC-IDS2017. Kľúčovým pozorovaním je diametrálne odlišné rozloženie chýb medzi MLP a LSTM: MLP Baseline zmeškal 528 útokov (FNR = 0.47 %), ale generoval 11 555 falošných poplachov (FPR = 2.54 %). LSTM Win=5 zmeškal 3 838 útokov (FNR = 9.10 %), ale generoval iba 581 falošných poplachov (FPR = 0.11 %) — teda 23× menej ako MLP Baseline.

Model	Epocha 1 (acc)	Epocha 15 (acc)	Val acc (best)	Čas/epocha
E1: MLP Baseline	0.937	0.979	0.981	∼5 s
E2: MLP Large	0.934	0.979	0.983	∼5 s
E3: LSTM Win=10	0.914	0.999	0.999	∼24 s
E4: LSTM Win=5	0.914	0.999	0.999	∼24 s

Tabuľka 13: Priebeh tréningu modelov na CIC-IDS2017 — per-day stratifikovaný split

Priebeh tréningu. Na rozdiel od globálneho chronologického splitu (kde val_loss LSTM rástol z 0.093 na 1.635 — príznak domain shiftu), per-day split zabezpečil stabilnú konvergenciu bez divergencie validačnej straty. Tréningová accuracy LSTM dosiahla 0.999 v epoche 15, čo je vyššie ako MLP (0.979) — LSTM s temporálnym kontextom lepšie zachytáva sekvenčné vzory v chronologicky usporiadaných dátach z každého dňa.

6.2 Výsledky na NSL-KDD

Testovacia množina obsahuje 22 543 vzoriek (9 710 Normal, 12 833 Attack). Na rozdiel od CIC-IDS2017, testovacia množina obsahuje typy útokov neprítomné v tréningovej množine, čo robí evaluáciu prísnejšou.

Model	Accuracy	Precision	Recall (Attack)	F1 (Attack)	Epochy
E5: MLP Benchmark	0.78	0.97	0.63	0.76	10
E6: LSTM Benchmark	0.77	0.97	0.61	0.75	10

Tabuľka 14: Výsledky na testovacej množine NSL-KDD (22 543 vzoriek)

MLP a LSTM dosiahli prakticky identické výsledky (ΔF1 = 0.01, ΔAccuracy = 0.01), čo potvrdzuje i.i.d. charakter NSL-KDD záznamov — temporálny kontext nepriniesol žiadnu merateľnú výhodu.

6.3 Súhrnné porovnanie všetkých experimentov

Exp.	Model	Dataset	Accuracy	F1	ROC-AUC	FNR
E1	MLP Baseline	CIC	0.98	0.9504	0.9985	0.47 %
E2	MLP Large	CIC	0.98	0.9490	0.9986	0.37 %
E3	LSTM Win=10	CIC	0.99	0.9477	0.9821	∼9 %
E4	LSTM Win=5	CIC	0.99	0.9455	0.9825	9.10 %
E5	MLP Benchmark	NSL-KDD	0.78	0.76	—	37 %
E6	LSTM Benchmark	NSL-KDD	0.77	0.75	—	39 %

Tabuľka 15: Súhrnné porovnanie všetkých šiestich experimentov (CIC: per-day split, 1. mája 2026)

6.4 Detailná analýza výsledkov

MLP Benchmark (E5) — NSL-KDD. MLP dosiahol Accuracy = 0.78 s výraznou asymetriou medzi triedami:

- **Precision(Attack)** = 0.97**:** Keď model predikuje útok, má takmer vždy pravdu — model sa naučil rozoznávať tréningové typy útokov s vysokou istotou.
- **Recall(Attack)** = 0.63**:** Approximately 37 % útokov ostáva nedetekovaných. Príčinou nie je slabá architektúra, ale špecifikum testovacej množiny NSL-KDD — obsahuje varianty R2L a U2R neprítomné v tréningovej množine.
- **Recall(Normal)** = 0.97**:** Legitímna komunikácia je identifikovaná spoľahlivo.

LSTM Benchmark (E6) — NSL-KDD. Po oprave off-by-one bugu vo funkcii `create_windows` dosiahol LSTM Accuracy = 0.77 — prakticky identický výsledok s MLP (ΔAccuracy = 0.01). Oba modely majú rovnakú Precision(Attack) = 0.97. Pôvodná bugovaná implementácia uvádzala LSTM Accuracy = 0.48, F1 = 0.49 — hodnoty blízke náhodnému hádaniu. Chyba spočívala v priradení mítky vzorke *za* oknom namiesto poslednej vzorky okna:

$$\text{(chybné)} \quad y[i+T] \quad \longrightarrow \quad \text{(opravené)} \quad y[i+T-1] \tag{29}$$

Generalizačný deficit — NSL-KDD.

$$\Delta\text{Acc}_{\text{MLP}} = 0.9948 - 0.78 = 0.2148, \qquad \Delta\text{Acc}_{\text{LSTM}} = 0.9824 - 0.77 = 0.2124 \tag{30}$$

Pokles o $\sim 21\,\%$ nie je overfitting, ale *out-of-distribution generalization* — testovacia množina NSL-KDD záámerne obsahuje útoky mimo distribúcie tréningových dát. Deficit je porovnateľný pre oba modely, čo potvrdzuje, že architektúra nie je limitujúcim faktorom — limitujúcou je povaha datasetu.

6.5 Interpretácia výsledkov

1. Metodológia rozdelenia dát determinuje výkon LSTM. Najvýznamnejším metodologickým zistením je, že spôsob rozdelenia datasetu má väčší vplyv na výkon LSTM ako voľba architektúry (tabuľka 9).

2. Temporálny kontext prináša hodnotu pri správnom splite. Po implementácii per-day splitu LSTM dosiahol F1 = 0.9477 — porovnateľné s MLP Baseline (0.9504, $\Delta = 0.003$). Temporálny kontext prináša merateľnú výhodu v podobe 23× nižšieho FPR (0.11 % vs. 2.54 %). Na NSL-KDD temporálny kontext nepomohol — záznamy sú i.i.d. bez chronologického poriadku.

Model	Precision	Recall	FNR	Praktická implikácia
MLP Baseline / CIC	0.91	1.00	0.47 %	Min. FNR, vyšší FPR (2.54 %)
LSTM Win=5 / CIC	0.99	0.91	9.10 %	Min. FPR (0.11 %), 23× menej poplachov
MLP Benchmark / NSL	0.97	0.63	37 %	Spoľahlivé pre známe útoky
LSTM Benchmark / NSL	0.97	0.61	39 %	Ekvivalentné s MLP na i.i.d. dátach

Tabuľka 16: Praktické implikácie Precision-Recall asymetrie

3. Precision-Recall asymetria a bezpečnostné implikácie. Na CIC-IDS2017 neexistuje jednoznačný víťaz — voľba závisí od bezpečnostných požiadaviek: MLP pre minimalizáciu FNR, LSTM pre minimalizáciu FPR.

4. Odporúčania pre produkčné nasadenie.

1. **Prostredia s nulovou toleranciou zmeškaných útokov** (*zero FN policy*): MLP Baseline — FNR = 0.47 %, jednoduchšia architektúra, rýchlejší tréning (5 s/epocha).
2. **Prostredia s obmedzenou kapacitou analýzy poplachov** (*alert fatigue*): LSTM Win=5 — FPR = 0.11 %, 23× menej falošných poplachov, F1 porovnateľné s MLP.
3. **Pre benchmarkové datasety (NSL-KDD typ):** MLP aj LSTM dosahujú ekvivalentné výsledky — preferovať MLP z dôvodu nižšej výpočtovej náročnosti.
4. **Klasifikačný prah:** znížiť pod 0.5 pre zvýšenie Recall — analýza v Precision-Recall krivkách.
5. **Per-day stratifikovaný split:** pre scenárové datasety (CIC-IDS2017 typ) nevyhnutná podmienka validného experimentu s LSTM.

7 Kritická analýza a diskusia

7.1 Porovnávanie architektúr

MLP — silné stránky a limitácie. MLP spracúva každý sieťový tok ako izolovanú vzorku $\mathbf{x} \in \mathbb{R}^{69}$, čím úplne ignoruje časový kontext medzi po sebe idúcimi tokmi. Napriek tomu dosiahol ROC-AUC = 0.9985 a FNR = 0.47 %. Dôvodom je charakter CIC-IDS2017: väčšina zastúpených typov útokov sa prejavuje extrémne odlišnými hodnotami štatistických príznakov aj v jednotlivých tokoch:

- **DoS/DDoS:** extrémne vysoké hodnoty `Flow Bytes/s`, `Packet Length Mean` a nízke `Flow Duration` — identifikovateľné v každom jednotlivom toku.
- **Brute Force:** vysoký počet `Fwd Packets/s` na špecifický port, charakteristické hodnoty `Flow Duration`.

- **Port Scan:** nízka hodnota `Flow Duration` s variabilným `Destination Port` — signatúra rozpoznateľná flow-by-flow.

Limitáciou MLP zostáva neschopnosť zachytiť viacfázové útoky rozložené cez desiatky tokov (Infiltration, pomalý Botnet), kde žiadna jednotlivá vzorka nevykazuje anomálne hodnoty. Tieto typy tvoria minoritu v CIC-IDS2017, čo vysvetľuje prečo MLP aj napriek tejto slepej škvrne dosiahol FNR $< 0.5\,\%$.

LSTM — vplyv per-day stratifikovaného splitu. Pri návrhu LSTM experimentov bolo nevyhnutné rozhodnúť o metodológii rozdelenia datasetu. CIC-IDS2017 je *scenárový dataset* — každý deň obsahuje odlišné typy útokov:

Deň	**Typy útokov**	**Globálny split**	**Per-day split**
Pondelok	iba BENIGN	Train	Train + Val + Test
Utorok	FTP-Patator, SSH-Patator	Train	Train + Val + Test
Streda	DoS Hulk, GoldenEye, Slowloris	Train	Train + Val + Test
Štvrtok	WebAttacks, Infiltration	Val	Train + Val + Test
Piatok	Botnet, PortScan, DDoS	Test	Train + Val + Test

Tabuľka 17: Rozdelenie typov útokov pri globálnom vs. per-day splite

Per-day stratifikovaný split — kvantitatívne výsledky. Per-day split z každého dňa vyčlenil 64 %/16 %/20 % riadkov v chronologickom poradí. Tým každá množina obsahuje záznamy zo všetkých dní:

Deň / súbor	**Train**	**Val**	**Test**
Monday (iba BENIGN)	338 867	84 717	105 897
Tuesday (FTP-Patator, SSH)	285 212	71 304	89 129
Wednesday (DoS, GoldenEye)	442 499	110 625	138 282
Thursday-Morning (WebAttacks)	108 947	27 237	34 047
Thursday-Afternoon (Infiltration)	184 572	46 144	57 679
Friday-Morning (Botnet)	122 183	30 545	38 183
Friday-Afternoon-DDos	144 455	36 113	45 143
Friday-Afternoon-PortScan	183 101	45 775	57 220
Spolu	**1 809 836**	**452 460**	**565 580**

Tabuľka 18: Per-day stratifikovaný split — počty riadkov z každého dňa

Výsledná distribúcia tried: Train 24.5 % útokov, Val 15.7 %, Test 7.5 %. Nižší podiel útokov v testovacej množine je spôsobený tým, že pondelok (iba BENIGN) prispieva výlučne legitímnymi tokmi.

Dôkaz divergencie pri globálnom splite. Priebeh tréningu LSTM Advanced (E3) s globálnym chronologickým splitom potvrdil domain shift:

Epocha	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Val Accuracy	Trend
1	0.168	0.093	0.961	konvergencia
2	0.084	0.067	0.974	konvergencia
5	0.050	0.281	0.921	val_loss začína rásť
7	0.005	0.931	0.789	divergencia
12	0.004	1.663	0.786	silný overfit
15	0.003	1.635	0.811	bez konvergencie

Tabuľka 19: Priebeh tréningu LSTM Advanced — divergencia strát pri globálnom splite

Tréningová strata klesla o 98 % (0.168 → 0.003), zatiaľ čo validačná strata vzrástla o 1 658 % (0.093 → 1.635). Model sa naučil rozoznávať DoS a Brute Force sekvencie s takmer dokonalou presnosťou, ale stratil schopnosť generalizovať na WebAttacks a Infiltration v testovacej množine.

Prečo náhodný split pre LSTM nie je riešením. Alternatívou by bolo použiť `shuffle=True` aj pre LSTM, čím by sa všetky typy útokov rovnomerne rozdelili. Tento prístup však eliminuje samotnú motiváciu pre LSTM: náhodne premiešané riadky nemajú temporálny kontext a okno 10 susedných vzoriek zachytáva 10 *náhodne vybraných* tokov z rôznych dní. Experimentálne overenie potvrdilo: LSTM s `shuffle=True` dosiahol ROC-AUC ≈ 0.5 a TP $= 0$ — model predikoval výhradne triedu Benign.

7.2 Analýza výsledkov ablačnej štúdie

E1 vs. E2 — vplyv kapacity MLP. MLP Large (E2) zvýšil počet neurónov $(64, 32) \rightarrow (128, 64)$ a Dropout $0.2 \rightarrow 0.4$ pri zachovaní všetkých ostatných podmienok. Namerané výsledky jednoznačne potvrdili H2 — a to silnejšie ako sa očakávalo:

$$\Delta\text{ROC-AUC} = 0.998582 - 0.998549 = +0.000033 \approx 0 \tag{31}$$

$$\Delta\text{F1} = 0.949043 - 0.950431 = -0.001388 \quad (\text{E2 mierne } \textit{horšie} \text{ ako E1}) \tag{32}$$

Model	ROC-AUC	Avg. Prec.	F1	Záver
E1: MLP Baseline	0.998549	0.994096	0.950431	referencia
E2: MLP Large	0.998582	0.994218	0.949043	F1 mierne horšie
Δ	+0.000033	+0.000122	−0.001388	H2 potvrdená

Tabuľka 20: Porovnanie MLP Baseline vs. MLP Large — ablačná štúdia kapacity

MLP Large nielen nedosiahol zlepšenie, ale vykazuje mierne *zhoršenie* F1 oproti Baseline. Väčší Dropout (0.4) nadmerne regularizuje sieť — zvýšenie počtu neurónov $(128, 64)$ bolo plne kompenzované silnejšou regularizáciou. Mierne zlepšenie FNR (0.37 % vs. 0.47 %, čo zodpovedá 120 menej zmeškaným útokom z 111 311) neprichádza za prijateľnú cenu: 1.97× väčší počet parametrov a mierne horšie F1. Záver: architektúra $(64, 32)$ dosiahla saturačný bod a je optimálna pre CIC-IDS2017.

E3 vs. E4 — vplyv dĺžky časového okna LSTM. Skrátenie okna z 10 na 5 záznamov pri per-day splite prinieslo prakticky identické výsledky:

Model	ROC-AUC	Avg. Prec.	F1	FNR
E3: LSTM Win=10	0.9821	0.9482	0.9477	∼9 %
E4: LSTM Win=5	0.9825	0.9450	0.9455	9.10 %
Δ	+0.0004	−0.0032	−0.0022	≈ 0

Tabuľka 21: Porovnanie LSTM Window=10 vs. Window=5 (per-day split)

Per-day split eliminoval hlavnú príčinu predchádzajúceho paradoxu (kde Window=5 vykazoval paradoxne lepšie F1 pri globálnom splite kvôli nižšiemu šumu): keďže temporálny poriadok je zachovaný vnútri každého dňa pre obe konfigurácie rovnako, dĺžka okna prestáva byť rozhodujúcim faktorom pre kvalitu sekvencií. Rozdiel $\Delta F1 = 0.0022$ zodpovedá ∼12 vzorkám z 565 575 testovacích. Pre produkčné nasadenie **odporúčame Window=5**: o 50 % menšia alokácia pamäte (≈ 2.5 GB úspora RAM pre $N = 1\,809\,836$) a nižšia latencia inferencie pri porovnateľnom výkone.

7.3 Analýza chýb

Kvantitatívna analýza FP a FN — CIC-IDS2017. Z bezpečnostného hľadiska nie sú všetky chyby rovnako závažné. False Negative (zmeškaný útok) reprezentuje neodhalené narušenie bezpečnosti — potenciálny únik dát, kompromitáciu systému alebo výpadok služieb. False Positive (falošný poplach) zbytočne zaťažuje administrátorov a môže viesť k ignorovaniu upozornení (*alert fatigue*).

Model	FN	FNR	FP	FPR
E1: MLP Baseline	528	0.47 %	11 555	2.54 %
E2: MLP Large	408	0.37 %	10 826	2.38 %
E3: LSTM Win=10	∼3 750	∼9 %	∼500	∼0.11 %
E4: LSTM Win=5	3 838	9.10 %	581	0.11 %

Tabuľka 22: Analýza chýb na testovacej množine CIC-IDS2017 — per-day split

MLP Baseline zmeškal 528 útokov z 111 311 — zachytil 99.53 % všetkých útokov. LSTM Win=5 zmeškal 3 838 útokov, ale generoval iba 581 falošných poplachov — oproti 11 555 pri MLP Baseline (23× menej). Z bezpečnostného hľadiska je voľba medzi modelmi otázkou prioritizácie: nižšie FNR (MLP) vs. nižšie FPR (LSTM).

Model	FN	FNR	FP	FPR
E5: MLP Benchmark	∼4 748	37.0 %	∼291	3.0 %
E6: LSTM Benchmark	∼5 003	39.0 %	∼291	3.0 %

Tabuľka 23: Analýza chýb na testovacej množine NSL-KDD

Kvantitatívna analýza FP a FN — NSL-KDD. Kľúčovým pozorovaním je extrémna asymetria medzi FNR a FPR pre oba modely: FPR ≈ 3 % (legitímna komunikácia je identifikovaná spoľahlivo), avšak FNR ≈ 37–39 % (každý tretí útok zmeškaný). Táto asymetria nie je spôsobená nedostatočnou kapacitou modelov, ale štruktúrou NSL-KDD: 37–39 % útokov patrí k typom (R2L, U2R varianty) neprítomným v tréningovej množine. Porovnanie s CIC-IDS2017 odhaľuje 79× zhoršenie FNR (0.47 % vs. 37 %) — ilustruje zásadné obmedzenie supervízovaného učenia pri out-of-distribution útokoch.

Prah θ	Recall (Attack)	FPR	Vhodné pre
0.3	$\nearrow$ (vyšší)	$\nearrow$ (vyšší)	Kritická infraštruktúra, nulová tolerancia FN
0.5	1.00	2.54 %	Štandardné nasadenie (MLP Baseline)
0.7	$\searrow$ (nižší)	$\searrow$ (nižší)	Prostredia s limitovanou kapacitou analýzy

Tabuľka 24: Vplyv klasifikačného prahu na Recall a FPR (MLP Baseline)

Optimalizácia klasifikačného prahu. Optimálna hodnota θ sa určuje minimalizáciou nákladovej funkcie $C = C_{\mathrm{FN}} \cdot \mathrm{FN} + C_{\mathrm{FP}} \cdot \mathrm{FP}$. Pre bezpečnostné systémy platí typicky $C_{\mathrm{FN}} \gg C_{\mathrm{FP}}$, čo odôvodňuje nižší prah. Táto analýza je vizualizovaná v Precision-Recall krivkách vygenerovaných pre každý model.

Typy útokov náchylné na FN. CIC-IDS2017:

- **Infiltration:** viacfázový útok s nízkoobjemovým prenosom — príznaky sa nelíšia od legitímnej komunikácie v jednotlivých tokoch. Identifikovateľný iba z temporálnej analýzy viacerých tokov.
- **Botnet (ARES):** pravidelná rytmická komunikácia s C&C serverom s nízkym objemom — identifikovateľná až z dlhodobých behaviorálnych vzoriek, nie z jednotlivých tokov.
- **Web Attacks (XSS, SQL Injection):** útok prebieha v rámci legitímnej HTTP komunikácie — štatistické príznaky tokov sa nelíšia od bežného webového prehliadania.

NSL-KDD:

- **R2L (Root to Local) — neznáme varianty:** útoky zneužívajúce vzdialený prístup s nízkym počtom paketov, ktorých profil príznakov sa podobá normálnej komunikácii. Tréningová množina obsahuje iba podmnožinu variantov.
- **U2R (User to Root) — neznáme varianty:** privilege escalation útoky s veľmi nízkym objemom prenosu. Extrémne nízka početnosť v tréningovej množine (menej ako 1 % vzoriek) spôsobuje, že model nemá dostatočnú tréningovú evidenciu.

Práve pre typy útokov s nízkoobjemovým prenosom a temporálnou rozptýlenosťou by LSTM s dlhším kontextovým oknom a datasetom s garantovaným chronologickým poriadkom mohol priniesť zlepšenie — avšak overenie tejto hypotézy presahuje rámec tejto práce a predstavuje hlavný smer ďalšieho výskumu.

7.4 Technické poznatky z HPC nasadenia

GPU infraštruktúra. Pôvodné pokusy o spustenie na lokálnom stroji zlyhali s chybou `Could not find cuda drivers` — tréning prebiehal výhradne na CPU pri rýchlosti $\sim 570\,\mu$s/krok. Po migrácii na HPC Perun (NVIDIA H200, 143 771 MB VRAM) a inštalácii balíkov `nvidia-cuda-*` cez pip (`tensorflow[and-cuda]`) GPU správne detekovaný:

```
GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Finálny run dosiahol rýchlosť MLP $\sim 556\,\mu$s/krok, LSTM ~ 3 ms/krok — rekurentné operácie sú menej paralelizovateľné ako maticové násobenia v Dense vrstvách (5.4× pomalší krok napriek rovnakej GPU). Celkový čas jedného kompletného behu všetkých štyroch experimentov: ∼15 minút (18:53–19:08).

Pamäťová náročnosť LSTM sekvencií. Transformácia 1 809 836 riadkov na sekvencie tvaru $(1\,809\,826, 10, 69)$ pri type `float32` vyžaduje:

$$18GB(Window = 10)/10(Window = 5)\,\text{GB RAM} \tag{33}$$

Na lokálnom stroji s 16 GB RAM táto alokácia spôsobila `Killed` signal od OS. H200 s 143 GB VRAM túto záťaž zvládol bez obmedzení. Toto potvrdzuje nevyhnutnosť HPC infraštruktúry pre LSTM experimenty na datasete tejto veľkosti.

Etapa vývoja	**ROC-AUC**	**F1**	**FNR**
Off-by-one bug + shuffle=False	≈ 0.500	≈ 0	$\approx 100\,\%$
Opravený bug + globálny chrono. split	0.9029	0.681	$\sim 21\,\%$
Opravený bug + per-day split (finálny)	**0.9821**	**0.9477**	$\sim$ **9** %
MLP Baseline (shuffle=True, bez bugu)	0.9985	0.9504	0.47 %

Tabuľka 25: Evolúcia výsledkov LSTM Advanced počas vývoja experimentov

Evolúcia výsledkov LSTM počas vývoja.

8 Obmedzenia a možnosti ďalšej práce

8.1 Obmedzenia

- **Offline dataset:** CIC-IDS2017 je statický — modely nie sú testované na schopnosť adaptácie na nové typy útokov (concept drift).
- **Binárna klasifikácia:** projekt nerieši viactriednu klasifikáciu (identifikáciu konkrétneho typu útoku), čo by bolo prakticky cennejšie.
- **Sekvenčné okná LSTM:** prístup predpokladá, že záznamy v CSV sú chronologicky zoradené vnútri každého súboru, čo nie je explicitne garantované (stĺpec `Timestamp` nebol použitý pri zoraďovaní).
- **Triedna nerovnováha v per-day splite:** nižší podiel útokov v testovacej množine (7.5 % vs. 19.7 % celkovo) je artefaktom pondelkového súboru (iba BENIGN), čo môže skreslif porovnanie s MLP testovacou množinou.
- **Absencia real-time evaluácie:** výkon v reálnom prostredí s nízkou latenciou nebol meraný.

8.2 Možnosti ďalšej práce

- **Viactriedna klasifikácia:** rozšírenie na identifikáciu konkrétneho typu útoku (DoS, DDoS, Brute Force...).
- **Transformer architektúra:** nahradenie LSTM modernými self-attention mechanizmami, ktoré lepšie spracúvajú dlhé sekvencie.
- **Autoencoder pre anomáliu detekciu:** namiesto supervízneho učenia trénovať len na legitímnom provoze a detekovať odchýlky.
- **Online learning:** adaptácia modelu na nové typy útokov bez úplného pretrénovania.

- **SHAP analýza:** identifikácia najdôležitejších príznakov pre interpretovateľnosť rozhodnutí modelu.
- **Optimalizácia klasifikačného prahu:** systematická analýza prahu θ minimalizujúca nákladovú funkciu $C_{\mathrm{FN}} \cdot \mathrm{FN} + C_{\mathrm{FP}} \cdot \mathrm{FP}$ pre konkrétne nasadenie.

9 Príspevok jednotlivých členov tímu

Člen tímu	Zodpovednosť	Konkrétne výstupy
Andrii Sysak	Predspracovanie dát, pipeline načítania	`data_preprocesing.py`, per-day split
Oleksandr Zemlyanskiy	Návrh a implementácia MLP a LSTM architektúr	`models.py`, `lstm.py`, oprava off-by-one bugu
Nikita Synenko	Tréning, evaluácia, vizualizácia výsledkov	`train_eval.py`, grafy, PR krivky
Andrii Rytskyi	Metodológia, ablation study, HPC nasadenie, správa	`main.py`, SLURM skripty, report, README

10 Zoznam použitej literatúry

Literatúra

[1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In ICISSP (pp. 108-116).

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

[4] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML.

[5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR, 15(1), 1929-1958.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825-2830.

[7] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI, 16, 265-283.